Mini Project Assignment: Medical Image Compression & Segmentation System

ISHIKA
230101303
B.TECH CSE(E) CORE

Assignment Tasks

Task 1: Image Compression

- Load a grayscale medical image (X-ray / MRI / CT)
- Implement Run Length Encoding (RLE)
- Calculate compression ratio and storage savings

```
    import cv2

import numpy as np


# Step 1: Load grayscale image

img = cv2.imread('img3.jpeg', 0)   # use your medical image


if img is None:

    print("Error: Image not found")

else:

    # Flatten image to 1D array

    pixels = img.flatten()


    # ----------------------------
```

```
# Step 2: Run Length Encoding
# ----------------------------
def rle_encode(data):
    encoding = []
    prev_pixel = data[0]
    count = 1

    for pixel in data[1:]:
        if pixel == prev_pixel:
            count += 1
        else:
            encoding.append((prev_pixel, count))
            prev_pixel = pixel
            count = 1

    encoding.append((prev_pixel, count))
    return encoding

encoded = rle_encode(pixels)

# ----------------------------
# Step 3: Compression Metrics
# ----------------------------
original_size = pixels.size          # number of pixels
compressed_size = len(encoded) * 2   # (value, count)
```

```
compression_ratio = original_size / compressed_size

space_saving = (1 - (compressed_size / original_size)) * 100


# ----------------------------

# Output

# ----------------------------

print("Original Size:", original_size)

print("Compressed Size:", compressed_size)

print("Compression Ratio:", round(compression_ratio, 2))

print("Space Saving (%):", round(space_saving, 2))
```

```
Original Size: 722016
Compressed Size: 1174060
Compression Ratio: 0.61
Space Saving (%): -62.61
```

Task 2: Image Segmentation

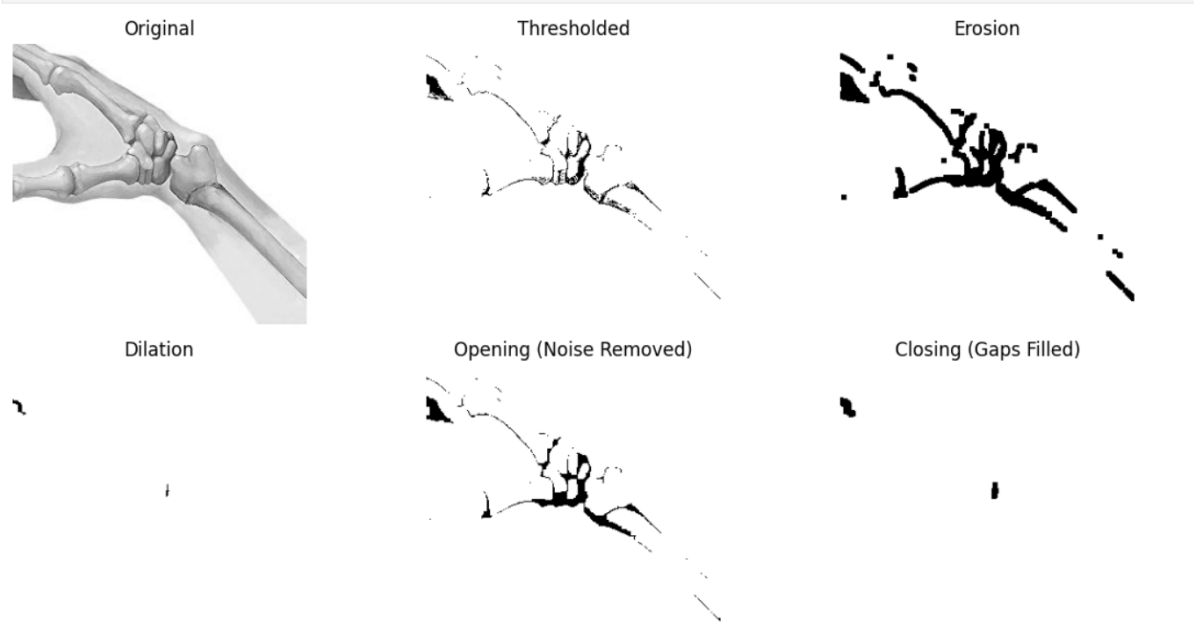
- Apply:
  - Global thresholding
  - Otsu's thresholding
- Identify regions of interest

```
import cv2

import numpy as np

import matplotlib.pyplot as plt
```

```
# Step 1: Read grayscale image
img = cv2.imread('img3.jpeg', 0)

if img is None:
    print("Error: Image not found")
else:
    # ----------------------------
    # Step 2: Thresholding (Segmentation)
    # ----------------------------
    _, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

    # ----------------------------
    # Step 3: Morphological Operations
    # ----------------------------
    kernel = np.ones((5,5), np.uint8)

    # Erosion (remove noise)
    erosion = cv2.erode(thresh, kernel, iterations=1)

    # Dilation (restore shape / fill gaps)
    dilation = cv2.dilate(thresh, kernel, iterations=1)

    # 🔥 Refinement (BEST PRACTICE)
    opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)   # erosion + dilation
    closing = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)  # dilation + erosion
```

```
# ----------------------------
# Display Results
# ----------------------------
plt.figure(figsize=(12,6))

plt.subplot(2,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(2,3,2)
plt.imshow(thresh, cmap='gray')
plt.title("Thresholded")
plt.axis('off')

plt.subplot(2,3,3)
plt.imshow(erosion, cmap='gray')
plt.title("Erosion")
plt.axis('off')

plt.subplot(2,3,4)
plt.imshow(dilation, cmap='gray')
plt.title("Dilation")
plt.axis('off')
```

```
plt.subplot(2,3,5)
plt.imshow(opening, cmap='gray')
plt.title("Opening (Noise Removed)")
plt.axis('off')

plt.subplot(2,3,6)
plt.imshow(closing, cmap='gray')
plt.title("Closing (Gaps Filled)")
plt.axis('off')

plt.tight_layout()
plt.show()
```



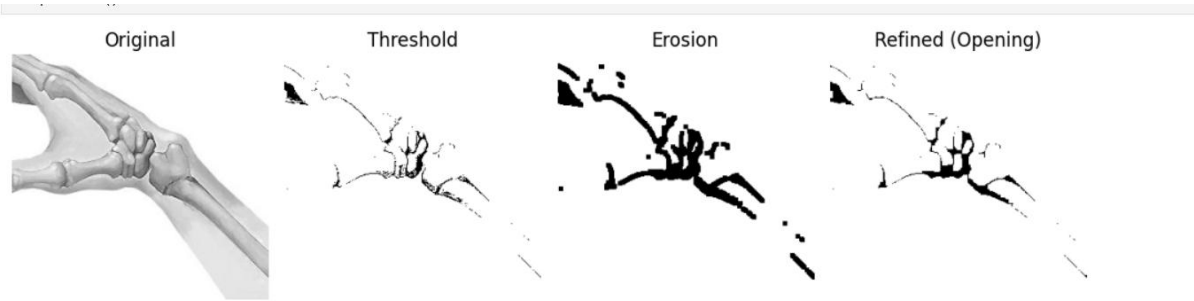
Task 3: Morphological Processing

- Apply:
  - Dilation
  - Erosion
- Refine segmented regions

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# Step 1: Read grayscale image

img = cv2.imread('img.jpg', 0)


if img is None:

    print("Error: Image not found")

else:

    # Step 2: Thresholding (segmentation)

    _, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)


    # Step 3: Kernel

    kernel = np.ones((5,5), np.uint8)


    # Step 4: Erosion

    erosion = cv2.erode(thresh, kernel, iterations=1)


    # Step 5: Dilation
```

```
dilation = cv2.dilate(thresh, kernel, iterations=1)

# 🔥 Step 6: Refinement (Opening = best for cleaning)
refined = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)

# ----------------------------
# Display
# ----------------------------
plt.figure(figsize=(10,5))

plt.subplot(1,4,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1,4,2)
plt.imshow(thresh, cmap='gray')
plt.title("Threshold")
plt.axis('off')

plt.subplot(1,4,3)
plt.imshow(erosion, cmap='gray')
plt.title("Erosion")
plt.axis('off')
```

```
plt.subplot(1,4,4)

plt.imshow(refined, cmap='gray')

plt.title("Refined (Opening)")

plt.axis('off')


plt.tight_layout()

plt.show()
```



Task 4: Analysis and Interpretation

- Compare segmentation results
- Discuss clinical relevance of extracted reg0ions

Thresholding gives rough segmentation with noise.
Erosion removes noise but reduces object size.
Dilation restores and expands regions.
Morphological operations like opening produce cleaner and more accurate segmentation.